Technical report — NLI over Swiss federal voting booklets with Apertus

What we built, how it works, and what the numbers say. Every number comes from a recorded run in `docs/runs/` (short names in brackets, listed under References) on the dev split, the validation set (val), the probe set or all three; **the sealed test dates have not been run**.

- **Track:** `Track 2A — OST: Multilingual Natural Language Inference over Swiss Official Voting Booklets`
- **Event:** Online
- **Project:** TODO project name
- **Team:** TODO team name — Rashad, Kaan (TODO full names)
- **Demo:** none

1. Summary

Given a claim in German, French or Italian and a source from an official Swiss federal voting booklet, the system decides whether the source supports (entailment, 0), does not settle (neutral, 1) or refutes (contradiction, 2) the claim. The source is a passage (task B) or a whole booklet PDF (task A). **Apertus v1.5 makes every decision.** Everything else is local and runs on CPU in the Docker image: reading the PDF, choosing the part of the booklet Apertus reads, and turning the paragraphs Apertus cites into evidence quoted verbatim with their 1-based PDF page. Apertus also names the part a claim refers to when the rules do not know the claim's opening, reads a booklet's contents when our parser cannot split it, and translates a claim for the keyword search of the fallback (section 4).

The central experiment, full booklet against selected context, decided the design: on 100 dev cases [E1-curve] the whole booklet scored 0.858 with 37,586 input tokens per case, the part the claim names (`section-route`) 0.989 with 1,248; without booklet text Apertus scores 0.435 [E2-cb].

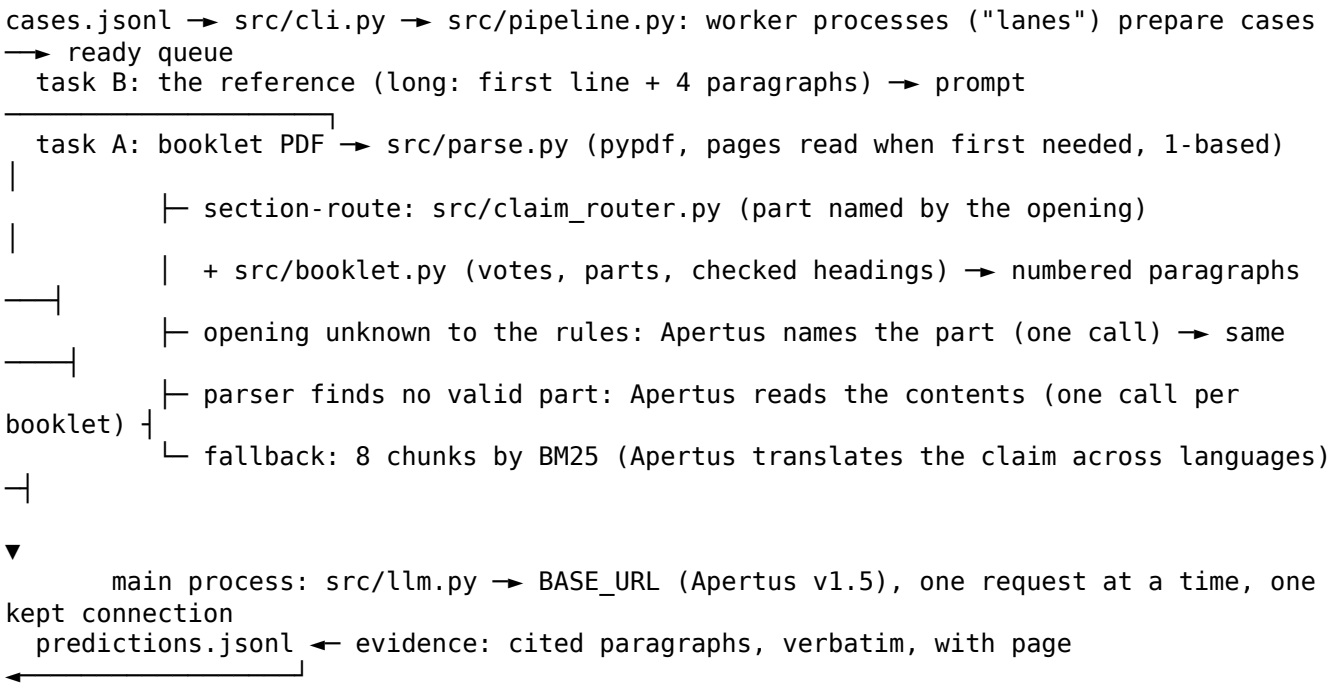
Final defaults, published image [FIN]	dev	val	probe (2 unseen dates)
Task A Macro-F1 (weight 0.30 of the score)	**0.973**	**0.969**	**0.981**
Task A evidence (0.12)	**0.985**	**0.965**	**0.985**
Task A tokens per case, input + output (K, 0.09)	1,251 + 14.8	1,225 + 14.7	1,172 + 16.6
Task A local time per case, model at 1.0 / 0.3 s per request (T, 0.09)	0.016 / 0.017 s (repeat: 0.015)	0.008 / 0.007 s	0.035 / 0.027 s
Task B Macro-F1 (0.28)	**0.987**	**0.968**	**0.977**
Task B tokens per case, input + output (K, 0.06)	815 + 8.8	820 + 9.4	789 + 8.9
Task B local time per case, 1.0 / 0.3 s per request (T, 0.06)	0.008 / 0.009 s	0.006 / 0.007 s	0.014 / 0.015 s
Labels written by Python (neutral fallback)	0 of 600	1 of 1,160	0 of 188
REF SCORE at 1.0 / 0.3 s per request (section 2)	**0.915 / 0.911**	**0.917 / 0.912**	**0.903 / 0.901**

Sealed test dates (2020-02-09, 2022-02-13, 2022-05-15): TODO, results after the organisers' evaluation.

2. What is scored, and what we optimise

The organisers' score (Q&A of 8 October; `docs/official_scoring.md`): **Score = 0.4 S_B + 0.6 S_A**, with S_A = 0.50 Macro-F1 + 0.20 evidence + 0.15 T + 0.15 K and S_B = 0.70 Macro-F1 + 0.15 T + 0.15 K. K = the best team's tokens per case ÷ ours; T = the best local time per case ÷ ours, where **local time is the container's wall-clock time minus the time during which a model request is in flight**: start-up, PDF reading, embedding and every gap between two requests count. Task A and task B run in separate containers. To rank our own versions we use **REF SCORE** (`scripts/official_score.py`): the same formula with "best" values fixed by us before any measurement (600 / 800 tokens, 0.05 / 0.005 s local time per case for task A / B). Local time is measured the organisers' way (`scripts/local_time.py`): the image with 2 CPUs and 4 GB against a fake model that replays saved answers after 1.0 s (about Public AI's speed), 0.3 s or 0.1 s per request.

3. Architecture



- `src/cli.py`: the entrypoint; each case is `prepare()` (local work) then `complete()` (model calls); every id is answered (neutral if all else fails), each answer written at once, exit code 0.
- `src/pipeline.py`: two forked lanes, each owning whole booklets, prepare cases while the main process sends them; one lane under 2.5 GiB. A **memory guard** (`src/rankers.py`) does without an embedding model whose need (e5: 1.5 GiB) exceeds the container's limit or that fails to load: long parts send their first 8 paragraphs, long task B references go whole. Caches are keyed by content; replaying the saved answers of all 1,180 dev and val requests reproduces every request, label and evidence (G1, G2).
- `src/llm.py`: the only module that calls a model (`BASE_URL`, temperature 0, one kept connection, retries).

3.1 How document context is prepared and supplied to the model

Parsing. pypdf reads a page's text when first needed, with its 1-based PDF page number; an unreadable page is repaired or left empty, never failing the booklet.

Selection (`section-route`, task A). (1) *Router:* the claims open with a formula naming their source ("Laut der Zusammenfassung …", "Le Conseil fédéral estime …"); regular expressions map it to one of five parts: summary, Federal Council, committee, text put to the vote, detailed explanation. If they find none, Apertus names the part (one call). (2) *Parser:* the table of contents gives each vote's sections (two layouts: before and after mid-2018); every start page is checked against its printed heading; the arguments are split by voice. If no part of the vote passes, Apertus

reads the contents pages (one call per booklet) and its start pages go through the same checks. (3) *Vote match:* the case's `vote` against the contents titles (fuzzy); if that fails, by meaning (model S). (4) *Paragraphs:* the part's lines grouped into paragraphs, each with its page; a part over 8,000 characters keeps its 8 paragraphs most similar to the claim (model S, a static multilingual embedding; e5 until session 13). (5) *Fallback:* no part, no vote, or any error → the booklet's 8 chunks of up to 1,000 characters with the best BM25 score for the claim's words; a claim in another language than the booklet is first translated into the booklet's language by Apertus (one call), and the reading call gets the claim as it is.

What the prompt contains. System: the instruction and the decision rule (`A-v4-section-route-L1`):

> *Decide in this order: 1. Does the reference text deal with the subject of the claim at all? If not, the label is 1 (neutral). 2. If it does: 0 if it supports the claim; 2 only if it states something that cannot be true together with the claim; otherwise 1. Missing information is never a contradiction. […]*

User: `PART:` (which part, whose voice), `REFERENCE TEXT:` with paragraphs `[1] … [n]`, `VOTE:`, `CLAIM:` as given. `response_format: json_schema` forces `{"paragraphs": [≤ 3 numbers], "label": 0|1|2}`. Task B sends the same rule (`v3-topic-first`) for `{"label": n}`; a reference over 3,000 characters is sent as its first line and the 4 paragraphs whose first 300 characters are most similar to the claim (e5).

Evidence. The paragraphs Apertus cited, verbatim with their page, then their halves, then (H, session 13) the paragraph before the first cited one when that one opens its page, and the cited page's whole text; at most five items, every one on its real page; none for neutral answers and task B.

4. How Apertus is used

The organisers (`docs/official_contract.md`): "Use only **Apertus v1.5** models (`swiss-ai/Apertus-v1.5-...`)" and "Every remote model call must use `BASE_URL`; local parsing, OCR, or embeddings may run locally." **Apertus makes every judgement; Python only decides what Apertus reads.** Python and the local embedding model read the PDF, pick the part, and copy what Apertus cites; they never set or change a label (the only Python label is the neutral fallback for an unreadable answer: 1 of 2,128 final labels). Model `swiss-ai/apertus-v1.5-8b` on Public AI, zero-shot, temperature 0, answers constrained by `json_schema`, only through `BASE_URL`; no fine-tuning.

Role of Apertus (final dev run [FIN])	Called when	Calls per 100 cases (A / B)	Tokens per call, in + out
Label and the paragraphs it rests on	every case	100 / 100	1,251 + 14.6 (A); 815 + 8.8 (B)
AP-part: the part a claim's opening names	the rules know no part for the opening	0 / –	≈ 236 + 49
AP-structure: read the booklet's contents	the parser finds no valid part	0 / –	≈ 1,825 + 192 per booklet
AP-translate: the claim into the booklet's language	fallback across languages	0.33 / –	141 + 55

Ablation, task A dev (minimum Macro-F1 0.60; task B 0.75)	Macro-F1	Evidence	Input tokens per case
nothing: claim and vote only [E2-cb]	0.435	—	≈ 200
8 chunks found by embedding, the early baseline [E5]	0.834	0.662	1,868
the whole booklet, 100 cases [E1-curve]	0.858	—	37,586
Apertus does all parsing too (part and contents, every case) [AP-all]	0.960	0.891	1,655 (+34 %; local time 8×)
the part the claim names (routed) [FIN]	**0.973**	**0.985**	1,251

Errors (session 12's final val answers, `scripts/error_types.py`): of 23 task A errors, **10 are routing errors** (the gold passage was not in what Apertus read; 8 in long law and detail parts cut to 8 paragraphs) and **13 reading errors**; all 18 task B errors are reading errors. Asked offline once per reading error (gold label given), Apertus names their kind: task A direction or negation 6, scope 5, number or date 2; task B another ballot or topic 6, number or date 6, scope 4, other 2.

Tried, not adopted (each by a rule fixed before its interleaved run; all in the code, off)	Result against today (Macro-F1, dev unless marked)	Why not
short prompts P-short (A), BP-short (B); three examples in the prompt (B)	0.670; 0.667; 0.933 vs 0.947	lower
K4: the part cut to its 4 most similar paragraphs	dev 0.990 (−35 % tokens); val 0.944 vs 0.959	fails on val
claim decomposition / tournament across languages / two-call negation	A 0.963 / 0.980 / 0.987 vs 0.983; B 0.937 / 0.970 / 0.960 vs 0.967	lower, 1.7–3 calls per case
L2 (second look with the 3 most similar paragraphs)	dev +0.010, val +0.003	below its +0.015 bar
vote-wide / booklet-wide fallback; B-para; B-cut4	0.927 / 0.857 (forced); 0.963 vs 0.967; −55 % tokens	fail their rules
task B selectors B-R / H / B-BM25	0.973 / 0.993 (+13 % tokens) / 0.967 vs 0.987	fail their rules
Apertus does all parsing (AP-all)	0.960 / 0.891 vs 0.977 / 0.980	lower, +34 % tokens
Python rule “neutral with a citation → contradiction”	val +0.014 offline; on-topic neutral 3 of 3 wrong	Python may not set a label
session 13 prompts A1–A5 against contradictions answered neutral	dev 0.902–0.983 vs 0.983; on-topic neutral 13–28 of 46 vs 32	fail on dev
A6: second look at a neutral answer that cites paragraphs, with their neighbours	val 0.973 vs 0.959 (9 fixes, 1 break), +0.8 % tokens	2 of 46 on-topic neutrals flipped (rule: at most 1)
B1 (ballots first) / B3 (T with the reference’s title) / B4 (claim first)	B1 void (unreadable); 0.980 / 0.980 vs 0.987 (B4: 5 unreadable)	lower or unreadable

5. Data

- **Dataset:** `OSTswiss/MNLIoverSwissVotingBooklets` v1.1 (commit `9ff08597`, MIT): 1,488 rows, 335 exact duplicates; each row gives one task A and one task B case.
- **Splits** (by voting date, duplicates removed): test 5 dates (267 rows); dev 300 rows; val 580 rows of the dev dates (never used to change a rule); **probe:** the 94 test rows of 2021-03-07 and 2025-09-28; three dates sealed.
- **Booklets:** the Federal Chancellery’s PDFs, downloaded by script and not committed. For parser coverage, every federal booklet from 2016 to 2026 in de, fr and it (93 PDFs; the sealed dates never downloaded). Swiss copyright law exempts official reports of authorities (Art. 5 URG). No personal data.

6. Evaluation

Method. The starter’s `evaluate.py`, unchanged. The endpoint drifts, so **only arms of one interleaved run compare** (every case answered by all versions back to back; all session 11 and 12 answers came from one backend). Rules were fixed before the runs; a version that fails stays in the code, switched off.

Parser coverage [COV] (93 booklets 2016–2026; the three languages of a date must agree): booklets whose votes all pass 64 → **88**, votes passing every check 178 → **244 of 249**, probe cases routed 83 → **90 of 94** (session 11).

Fallbacks, forced [FB-dev, FB-val, S12-dev, S12-val]: every case put into a fallback situation by a test-only switch (“no part”: the router returns nothing, AP-part off; “no vote”: the vote match fails; “no parser”: none of the vote’s parts passes its checks). Macro-F1 / evidence, dev; val (balanced 300); each line against the arm of its own interleaved run:

Situation, version	dev	val	Tokens (dev)	Adopted?
routed (today, not forced)	0.977 / 0.980	—	1,238 + 14.5	
no part: e5’s 8 chunks of the booklet (before session 12)	0.900 / 0.672	0.875 / 0.588	1,868 + 15.8	replaced

Situation, version	dev	val	Tokens (dev)	Adopted?
no part: BM25 chunks, Apertus translates across languages	**0.956 / 0.736**	**0.957 / 0.755**	2,591 + 44.3	**yes**
no parser: e5's chunks (before session 12)	0.896 / 0.672	0.872 / 0.593	1,868 + 15.8	replaced
no parser: Apertus reads the contents	**0.946 / 0.925**	**0.939 / 0.917**	1,564 + 42.8	**yes**

Not adopted: vote-wide (0.927 / 0.873) and booklet-wide paragraphs (0.857 / 0.856); model S's chunks (0.913, dev).

Task B, fewer tokens [TB-dev, TB-val]: a reference over 3,000 characters is cut to its first line and the 4 paragraphs whose first 300 characters are most similar to the claim (T): Macro-F1 dev 0.987, val 0.968 with 815 input tokens per case, against 0.967, 0.954 with 1,994 for the whole reference.

Local time (the organisers' measure; image, 2 CPUs, fake model replaying the final answers) [FIN]. What is local in task A is PDF reading (about 1.3 s per booklet, pure-Python pypdf) and the ranking of long parts. With e5 (until session 12) the ranking took about 3 s per long part and stopped hiding behind fast requests; model S (session 13) ranks in milliseconds:

Final image, local time per case	1.0 s per request	0.3 s	0.1 s
Task A, dev, model S (session 13)	0.016 s	0.017 s (repeat 0.015 s)	0.025 s
Task A, dev, e5 (session 12's final image)	0.017 s	0.160 s (repeat 0.113 s)	0.364 s
Task B, dev (e5 for selector T)	0.008 s	0.009 s	0.132 s

Single measurements at 0.3 s vary by up to about ±0.03 s.

Memory [GRID, GUARD]: with model S, task A peaks at 206 MiB (session 12 with e5: 2,287 MiB); task B at 1,164 MiB (e5 for selector T); one lane under 2.5 GiB; below 1.5 GiB no e5 (long task B references go whole). Through the final image, both tasks write every response at 4g, 2g, 1.5g, 1g, 768m and 512m, with 2 CPUs and with 1.

Probe: two unseen voting dates (test rows of 2021-03-07 and 2025-09-28, opened in session 10). Through the final image [FIN]: task A **0.981**, evidence **0.985** (64 of 65), task B **0.977**; REF SCORE **0.903 / 0.901**. Session 12's image: 0.971, 0.969, 0.977. 90 of 94 task A cases are routed; the 4 others are claims about a committee in votes that have none, answered by the BM25 fallback.

7. Limitations

- **Tuned on dev.** Prompts, router patterns and parser checks were written with dev in view; val confirms; the probe (two unseen dates) lands at val's level.
- **`section-route` depends on the booklet layout** (2016–2026 checked: 244 of 249 votes pass). A claim whose opening neither the rules nor Apertus place, or a vote the parser cannot read, falls back (answered, less accurate).
- **Endpoint and speed.** Our scores hold for the backend that answered; with a fast model, PDF reading no longer hides behind the requests (model S removed e5's share in session 13).
- **Evidence pages.** If the official run checks the *first* page of the gold passage, about 40 % of our hits would not count (the passage often starts a page before the paragraph Apertus cites); H

recovers part of it (§ 4 of the session 13 report; `docs/organiser_questions.md`).

- **Not political advice.** The system checks a claim against the booklet's text only. Its answers are not voting recommendations. Every task A label 0 or 2 comes with the booklet page and the verbatim text it rests on.

8. Reproducibility

- **Published image:** `ghcr.io/rashad18236/hackapertus-voting-nli:latest` (built from `main` and tested on a fresh runner by `.github/workflows/publish-image.yml`), run with the organisers' command (README); this report's numbers come from `sha-54032b4a7970` (session 13 branch).
- **From source:** `make run` builds the image and answers `examples/cases.jsonl`.
- **Hardware:** CPU only; 2 CPUs and 4 GB measured (peak 0.2 GiB task A, 1.2 GiB task B); the image answers every case down to 512 MiB, with 2 CPUs or 1: below 1.5 GiB task B without e5 [GRID, GUARD].
- **Determinism:** temperature 0; a fake model replays saved answers by request hash (G1, G2); 347 tests.

License

Code: Apache-2.0 (the repository's `LICENSE`). Report text: Creative Commons Attribution 4.0 (CC-BY-4.0).

References

- Dataset: OSTswiss/MNLIoverSwissVotingBooklets v1.1, Hugging Face (MIT). Starter: `gitlab.com/ifsoftware/hackapertus-starter`. Apertus v1.5, Swiss AI Initiative.
- Wang, L. et al. (2024). *Multilingual E5 Text Embeddings.* arXiv:2402.05672 (MIT). Model S: `static-similarity-mrl-multilingual-v1` (Apache-2.0). Robertson, S., Zaragoza, H. (2009). *BM25 and Beyond.*
- Runs (`docs/runs/`, dated 2026-10-08 to 14): FIN `final-s13_*`; S13 `prompts-A_*`, `prompts-B_*`, `S-val_valA580`, `combination-H-S_valA580`, `evidence-pages_offline`, `error-types_valAB`; GRID `memory-grid-final-s13_grid24`; S12 `S-and-fallbacks_devA300`, `bm25ap-fallback_valA300`, `ap-structure-ask_*`; GUARD `memory-guard_grid48`; TB `taskb-selectors_*`; COV `parser-coverage_booklets93`; FB `fallbacks_*`; E1-curve `context-curve_devA100`; E2-cb `closed-book_devA300`; E5 `section-route-vs-embed_devA300`; AP-all `ap-all_devA300`.